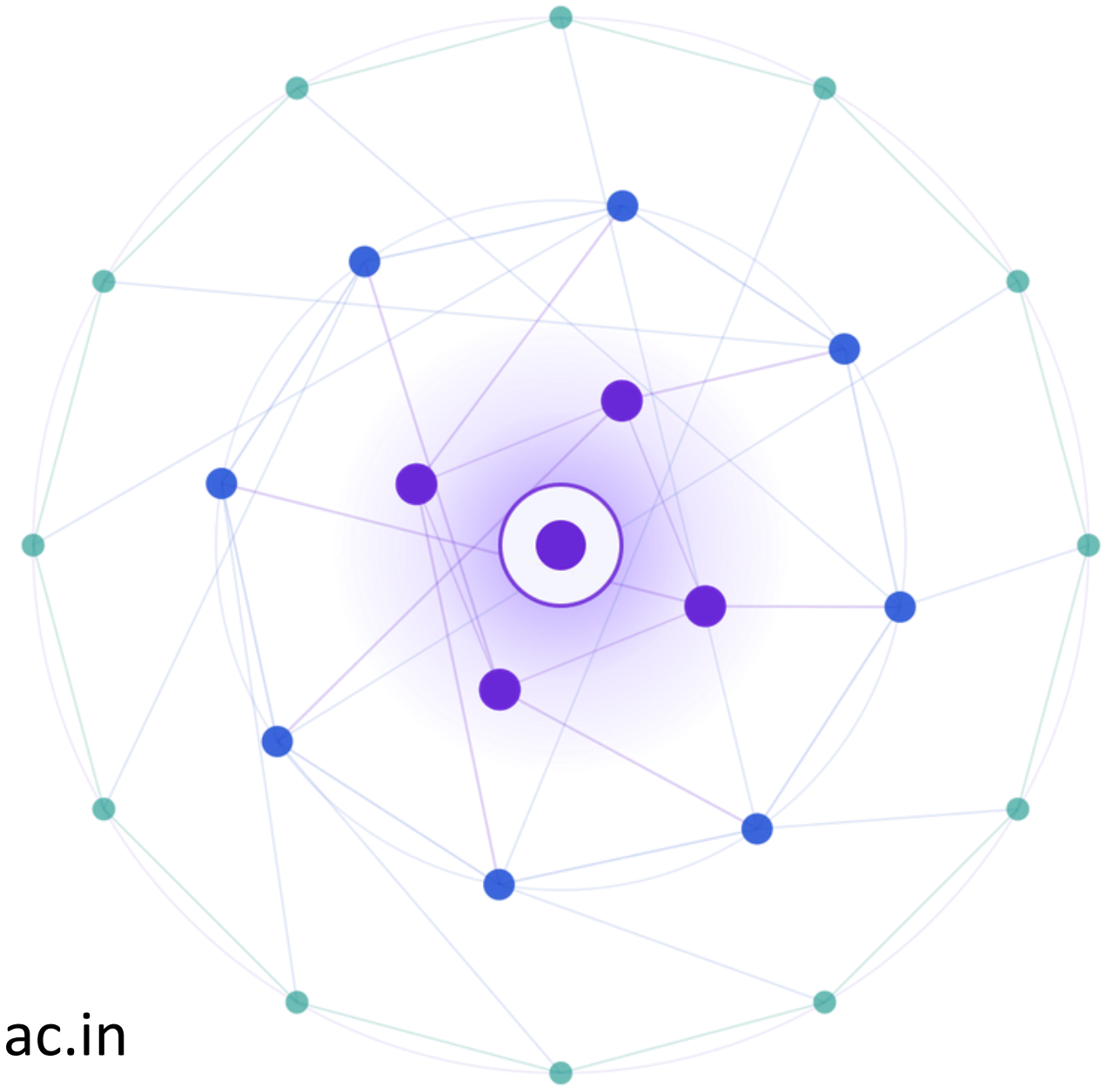


SAMSUNG PRISM

Generative AI Hackathon

3rd Edition 2026 – 27

- Theme ID - 02 – Smart Guided Troubleshooting Engine (TroubleIR)
- Team Name - Commit Happens
- College Name - VIT University
- Member Name & Email 1- Parth Garg | parth.garg2024@vitstudent.ac.in
- Member Name & Email 2- Sahil Sadhwani | sahil.sadhwani2024@vitstudent.ac.in
- Member Name & Email 3- Yashwanth | gajula.yashwanth2024@vitstudent.ac.in
- Submission Github link - [Github Link](#)



Theme

Galaxy users describe symptoms in everyday language, while the device needs a precise troubleshooting flow tied to a Settings screen.

Example complaint

“My screen is too bright at night.”

Expected response

- Identify the likely display issue
- Retrieve relevant support guidance
- Return ordered actions and step groups
- Attach a voiceassist:// deeplink where the plan has one
- Return no_match when the pipeline has no SLS context or cannot extract a plan

Project focus

TroubleIR turns support content into a schema-shaped API response that a guided troubleshooting experience can render.



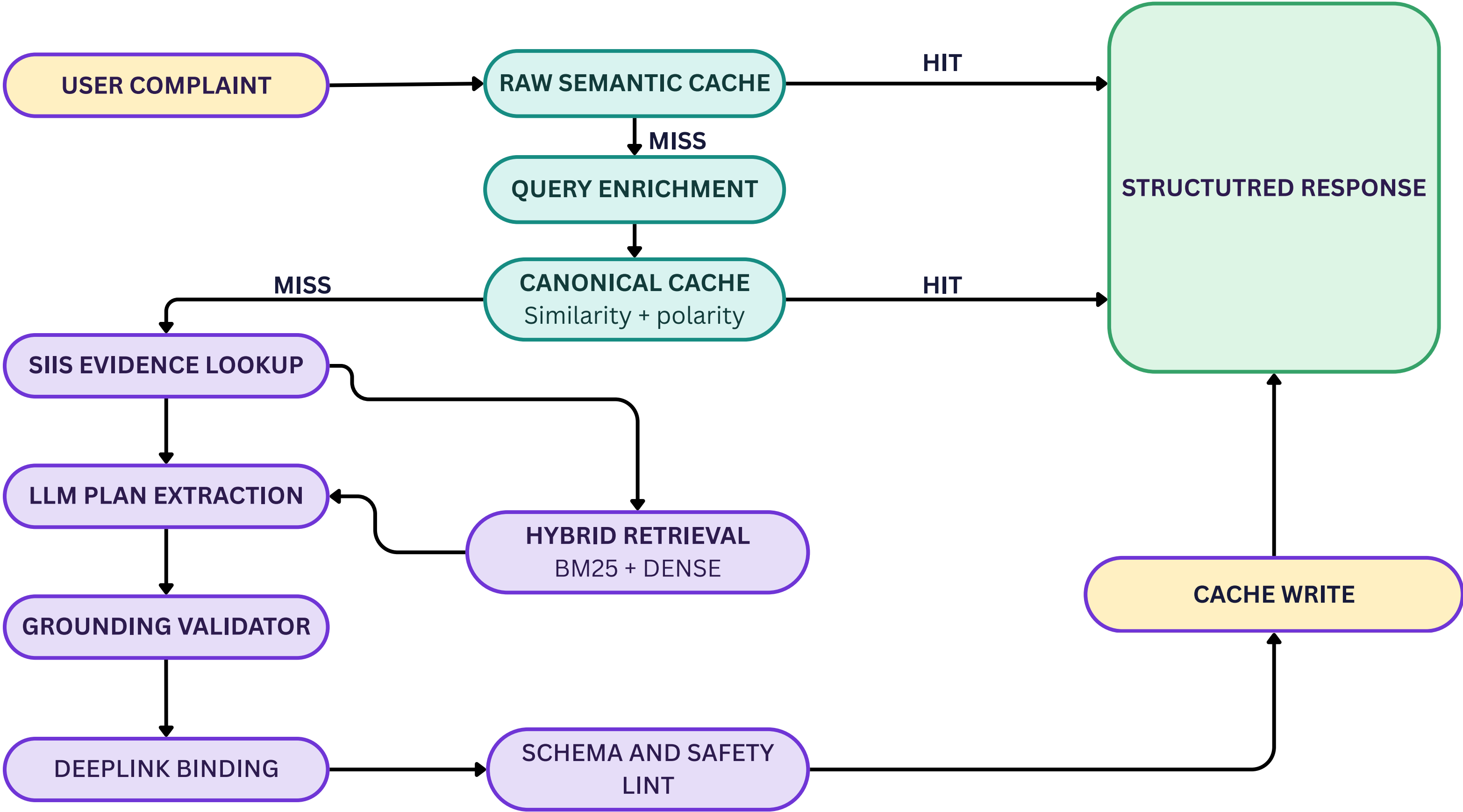
Existing Solutions & Gaps

Existing approach	Gap for guided troubleshooting
Keyword search in support articles	The user still has to interpret long text and find the relevant setting.
A general chat response	Free-form text does not guarantee the required response structure or a usable deeplink.
Semantic cache alone	Similar wording can express opposite intent, such as a screen that is too bright versus too dim.
A retrieved Settings category	A category can be broader than the screen where the action happens.

What TroubleIR adds in the prototype

- Hybrid BM25 and dense retrieval for SIIS articles and deeplink catalog entries
- A Pydantic response schema and a separate lint pass for URI format, web URL leaks, and action order
- A cache lookup with a similarity threshold and polarity-token check
- A no_match response when no SIIS context or no extracted plan is available

Our Solutions & Architecture Diagram



Demo & Product Walkthrough

Demo input

“Screen is too bright at night”

What the demo shows

- The user submits a complaint to POST /v1/troubleshoot
- The service generates a canonical query and paraphrase variations
- On a cache miss, it ranks an SIIS article and asks the configured Groq model for structured JSON
- The service normalizes actions into the Pydantic response schema and attaches deeplink metadata when available
- The frontend displays action cards, Settings path visualization, response metadata, and a JSON download control

Additional API surfaces

- Health check
- Deeplink catalog search and pagination
- Settings screen index for visualization
- Protected cache statistics and cache invalidation endpoints when DEBUG_TOKEN is configured

Tools and tech stack used

Layer	Technology in the repository
API	Python, FastAPI, Uvicorn
Structured data	Pydantic v2 and JSON
LLM calls	Groq SDK with the model selected by MODEL_NAME
Retrieval / cache embeddings	Sentence Transformers
Sparse retrieval	rank-bm25
Cache	In-memory cache, with optional Redis backend
Validation	Custom grounding and lint modules, plus pytest tests
Demo UI	HTML, GSAP, and Lenis
Deployment option	Gunicorn configuration for multiple Uvicorn workers

Samsung-provided project assets used by the code

- SIIS response dataset
- Deeplink catalog
- Required response schema reference

Impact & Use case

Primary use case

A Galaxy user reports a symptom without knowing the setting name or navigation path.

Example scenarios covered by evaluation data

- Battery drains quickly after an update
- Screen is too bright at night
- Navigation gestures behave unexpectedly
- Screen flickers while battery drains
- Out-of-scope complaints should return no_match

Value of the structured response

- Gives a UI a consistent goal, action, and step-group format
- Preserves the distinction between automatic, manual, and critical actions
- Carries deeplink data alongside the written steps
- Gives the client diagnostic metadata such as cache-hit status and latency

Innovation Highlights, Evidence and Limitations

Implemented highlights

- Hybrid retrieval: BM25 + dense embeddings for support text and deeplink ranking
- Polarity-aware cache check before reusing a similar cached plan
- Response checks: Pydantic model + lint module for URI format, ordering, URL leaks
- Traceable grounding: IDF-weighted overlap score per action step vs. SIIIS text
- Dependency tracking: cache entries linked to deeplinks, with an invalidation endpoint

Evidence & correctness signal

- 578 deeplink entries · 20 SIIIS articles
- 5 evaluation cases; 0 no_match cases in results.jsonl
- 1 polarity test for opposite-intent queries; rejection is not asserted
- results.jsonl contains 5 records with 10 query variations each
- 38 test functions across 5 pytest modules (schema, cache, lint, grounding, API)
- **Measured latency:**
 - raw cache hit p50 42 ms, p95 180 ms;
 - canonical cache hit p50 1,100 ms, p95 2,400 ms;
 - cold path p50 7,800 ms, p95 14,200 ms

Current limitations

- Grounding failures produce diagnostics but do not yet block a returned plan
- SettingsGraph supports screen-path visualization; the orchestrator does not yet use it as a rejection gate
- voiceassist://dummy_positive is used when no catalog screen maps cleanly

What's next

- Enforce grounding coverage and confidence thresholds, returning `no_match` when a plan does not meet them
- Use the `SettingsGraph` to reject deeplinks that resolve only to parent menus
- Replace the `dummy_positive` fallback with an explicit policy for missing catalog mappings
- Run and publish comparisons against raw LLM output and retrieval plus extraction
- Strengthen cache tests for opposite-intent queries, including an assertion that verifies rejection
- Add tests for compound complaints and catalog updates



Brownie points slide(differentiation)

TroubleIR treats troubleshooting as structured compilation.

The system does more than generate support text. It transforms a complaint into a typed response containing goals, ordered actions, step groups, and navigation metadata.

Why this is useful to a Samsung troubleshooting experience

- A frontend can render the returned plan consistently because the response has a fixed schema
- The cache checks intent polarity before reusing a similar request
- The lint pass identifies malformed deeplinks, leaked web URLs, and unsafe action ordering
- The API returns diagnostics that make behavior easier to inspect during development and evaluation

Honest differentiator

The prototype combines retrieval, structured extraction, cache checks, and response validation in one runnable flow. Its next engineering step is to make the existing grounding and screen-resolution signals hard delivery gates.

Checklist- Updated on Public GitHub

- Working prototype code —GitHub repository — Yes | [GithubLink](#)
- README with reproducible setup instructions — Yes
- Demo video, maximum 5 minutes — Yes | Link: [DriveLink](#)
- Presentation file in PPT or PDF — Yes

Thank you

TroubleIR demonstrates a structured approach to converting natural-language troubleshooting requests into a Samsung-style response that a guided UI can use.

Organised by the Language AI Team and the PRISM Team, Samsung R&D Institute India

Team Contact E-mail: parth.garg2024@vitstudent.ac.in